

Software Project Plan

Traffic Prediction and Analysis System

Assignment No. 7

Group: E-16

Members: Harshal Jaiswal
Pranav Jagtap
Suyash Jobanputra
Shreyash Jobanputra

1. Problem Statement

Prepare a detailed software project plan for the development of a **Traffic Prediction and Analysis System**. The project plan should incorporate scheduling techniques such as Gantt charts and PERT charts to map out the development timeline, visualize task dependencies, and determine the critical path of the project.

2. Abstract

Effective project planning forms the backbone of any successful software development endeavor. It encompasses task identification, effort estimation, dependency mapping, and continuous progress tracking. This document presents a comprehensive project plan for the **Traffic Prediction and Analysis System** — a full-stack web application that leverages real-time data, historical patterns, and machine learning to forecast urban traffic congestion. The plan employs a Work Breakdown Structure (WBS), a Gantt chart for timeline visualization, and a PERT chart for dependency and critical path analysis, collectively providing a structured roadmap for the development lifecycle.

3. Introduction

Modern software systems demand rigorous planning to navigate the complexities of multi-module architectures, third-party integrations, and iterative feedback cycles. A well-defined project plan mitigates risks such as schedule overruns, scope creep, and inefficient resource allocation.

The **Traffic Prediction and Analysis System** addresses the challenge of urban traffic congestion by integrating data from Google Maps Directions API, OpenWeatherMap API, and Calendarific API with historical traffic records, construction and diversion updates, event schedules, and citizen-reported infrastructure defects. The system generates a weighted traffic score (0–100) for any given route and time slot, empowering commuters to make informed travel decisions.

Given the system's reliance on multiple data sources, a machine learning module (MobileNet for pothole detection), and a gamification engine for citizen engagement, meticulous scheduling across design, development, integration, and testing phases is essential.

3.1 Planning Objectives

The key objectives driving this project plan are:

- i) Decompose the project into well-defined, manageable tasks.
- ii) Estimate realistic durations for each development activity.
- iii) Map inter-task dependencies to identify sequencing constraints.
- iv) Construct a visual project schedule using industry-standard tools.
- v) Highlight the critical path to focus monitoring efforts.
- vi) Anticipate risks and establish mitigation strategies.

4. Development Phases

The project follows a phased software development approach. The lifecycle is divided into eight distinct phases, each contributing a critical deliverable:

#	Phase
1	Requirement Analysis
2	System & Architecture Design
3	Database Schema Design
4	Backend Development
5	Frontend Development
6	API & ML Integration
7	Testing & Quality Assurance
8	Deployment & Documentation

5. Work Breakdown Structure

The WBS decomposes the project into discrete tasks with estimated durations and explicit dependency chains.

ID	Task	Duration	Depends On
T1	Requirement Analysis	2 Weeks	—
T2	System & Architecture Design	2 Weeks	T1
T3	Database Schema Design	2 Weeks	T1
T4	Backend Development	4 Weeks	T2, T3
T5	Frontend Development	3 Weeks	T2, T3
T6	API & ML Integration	3 Weeks	T4, T5
T7	Testing & Quality Assurance	2 Weeks	T6
T8	Deployment & Documentation	2 Weeks	T7

5.1 Task Descriptions

T1 — Requirement Analysis (2 Weeks): Involves stakeholder interviews, feature prioritization, and preparation of the IEEE 830 SRS document. Core requirements include traffic prediction, pothole reporting with ML verification, citizen gamification, and an admin dashboard.

T2 — System & Architecture Design (2 Weeks): Defines the MVC architecture pattern using Node.js/Express for the backend and React.js for the frontend. Includes API endpoint specification, service layer design, and interaction diagrams for the prediction engine and external API modules.

T3 — Database Schema Design (2 Weeks): Creates 19+ MongoDB collections covering Users, PathInfo (traffic scores), Complaints, Images (potholes), Events, Construction zones, Diversions, Schools, Hospitals, Hotels, Malls, HotspotLocations, PredictionLogs, and TrafficStatus records.

T4 — Backend Development (4 Weeks): Implements JWT-based authentication, the weighted traffic prediction engine (7 factors: Historical 25%, Weather 15%,

Construction 15%, Events 20%, Hotspots 12%, Potholes 8%, Complaints 5%), the automated cron-based data collector, gamification scoring, and 22 REST API controllers.

T5 — Frontend Development (3 Weeks): Builds the React.js single-page application with interactive traffic maps (Google Maps integration), prediction result cards, pothole image upload with camera capture, citizen score leaderboard, and the admin management dashboard.

T6 — API & ML Integration (3 Weeks): Connects frontend to backend APIs. Integrates Google Maps Directions, OpenWeatherMap, and Calendarific external services. Deploys the MobileNet pothole detection model and configures Cloudinary for image storage.

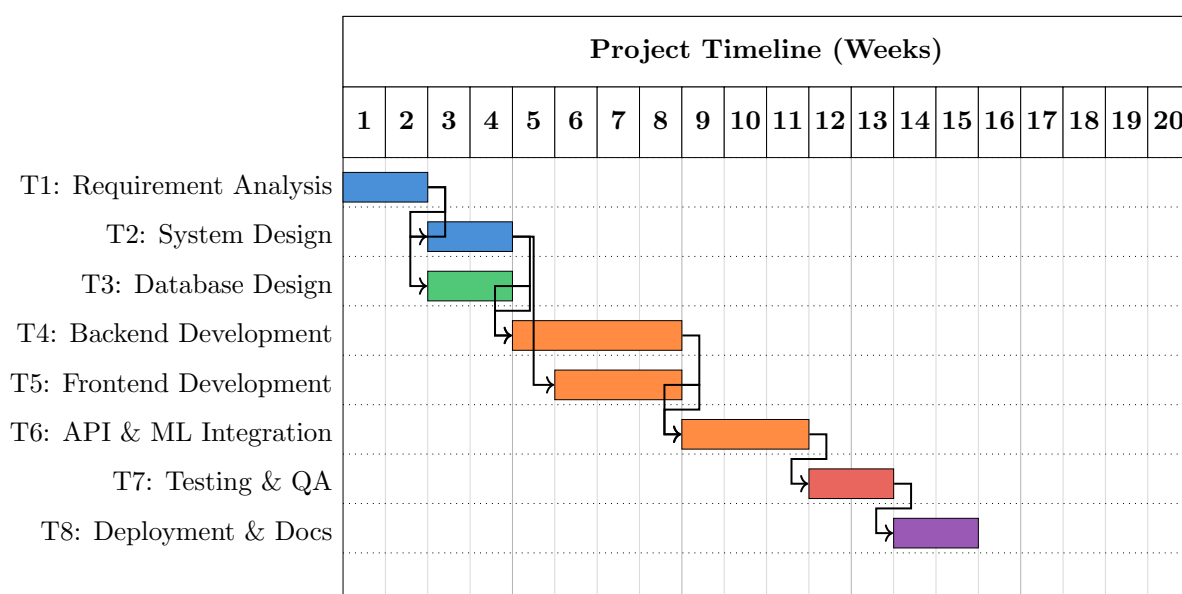
T7 — Testing & Quality Assurance (2 Weeks): Unit tests for individual modules, integration tests for API-to-database flows, and User Acceptance Testing (UAT) with performance benchmarking of the prediction engine under concurrent load.

T8 — Deployment & Documentation (2 Weeks): Production deployment on Render (backend) and Vercel (frontend). Finalizes blackbook documentation, user guide, and project handover materials.

6. Project Scheduling

6.1 Gantt Chart

The Gantt chart below maps each task against a 20-week timeline, with color-coded bars indicating the project phase and arrows denoting task dependencies.



Legend: █ Planning █ Database █ Development █ Testing █ Deployment

Figure 1: Gantt Chart — Traffic Prediction and Analysis System

6.2 Gantt Chart Analysis

The timeline reveals that T2 (System Design) and T3 (Database Design) execute **concurrently** after T1, reducing the schedule by 2 weeks compared to sequential execution. Backend (T4) and Frontend (T5) development also overlap during weeks 6–8, enabling parallel workstreams across the team. The integration phase (T6) acts as a synchronization point, beginning only after both T4 and T5 are complete. The total planned duration spans **15 active weeks** with 5 weeks of buffer within the 20-week window.

6.3 PERT Chart

The PERT diagram represents tasks as nodes with directed edges indicating dependencies. The **critical path** is highlighted in red.

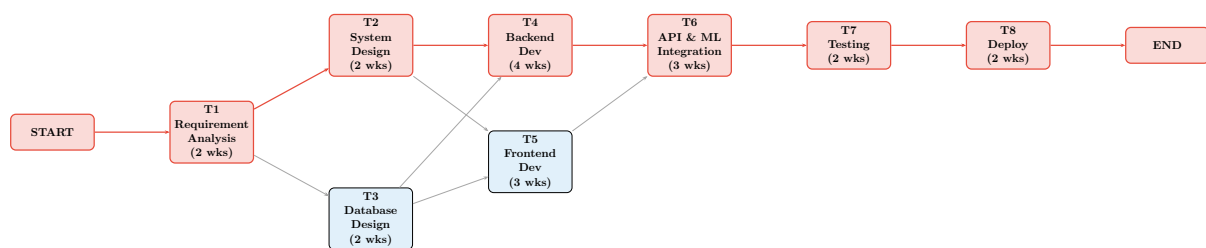


Figure 2: PERT Chart — Task Dependencies and Critical Path

6.4 Critical Path Identification

The critical path comprises the longest chain of dependent tasks:

T1 → T2 → T4 → T6 → T7 → T8

Task	Duration	Cumulative
T1: Requirement Analysis	2 wks	2 wks
T2: System Design	2 wks	4 wks
T4: Backend Development	4 wks	8 wks
T6: API & ML Integration	3 wks	11 wks
T7: Testing & QA	2 wks	13 wks
T8: Deployment	2 wks	15 wks

Any delay in a critical-path task directly extends the project deadline. Non-critical tasks T3 (Database Design), and T5 (Frontend Development) carry **2–3 weeks of float**, meaning minor delays in these tasks will not impact the overall schedule.

7. Risk Assessment

Risk	Description	Mitigation
API Limits	Google Maps / Open-WeatherMap impose daily rate caps	15-min response caching
ML Accuracy	MobileNet pothole detection may produce false positives	Confidence threshold ≥ 0.85 + admin review
Data Gaps	Limited historical traffic data for newly added routes	Backfill scripts + city-average fallback
Integration	Frontend-Backend API contract mismatches	Shared API specification document
Performance	Slow response under concurrent predictions	Database indexing + query optimization
Availability	Team bandwidth reduced during exams	5-week slack in non-critical tasks

8. Conclusion

This document presented a structured project plan for the Traffic Prediction and Analysis System. The Work Breakdown Structure decomposed the project into eight manageable tasks, the Gantt chart provided a visual timeline with concurrent workstreams, and the PERT chart identified a 15-week critical path through the project lifecycle.

By leveraging these industry-standard project management techniques, the development team can systematically track progress, allocate resources efficiently, and deliver the system within the planned schedule while maintaining quality standards.